

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Звіт до практичного заняття №4

«Технології програмування на C/Embedded»

Прийняв

Кондратенко Д. Ю.

“8” вересня 2026 р.

Виконав

Студент групи ІП-41

Чередніченко Артем

Київ – 2026

Практичне заняття №4

Тема: Динамічні структури даних. Однозв'язний список.

Мета: Мета роботи – набути практичних навичок роботи з динамічною пам'яттю та структурами, навчитися створювати однозв'язний список, заповнювати його, виконувати обхід і коректно звільняти пам'ять.

Завдання: Створити додаток, який виводить на екран інформацію про серію книг про Гаррі Поттера. Інформація про книги повинна міститися у зв'язаному списку. Кожен елемент повинен містити наступну інформацію: назва книги, ціна, число сторінок, мова, вага, рік видання.

Звіт виконання – додати:

- 1) посилання на додаток у вашому Git репозиторії;
- 2) скріншот виконання додатку.

Виконання

Додаток розроблено мовою C у середовищі CLion із системою збирання CMake. Дані про книги зберігаються в однозв'язному списку, елемент якого описано структурою:

```
typedef struct Book {  
    char    title[MAX_TITLE];    /* назва книги          */  
    double  price;               /* ціна, грн            */  
    int     pages;               /* число сторінок       */  
    char    language[MAX_LANG]; /* мова                 */  
    double  weight;              /* вага, г              */  
    int     year;                /* рік видання          */  
    struct Book *next;           /* наступний елемент */  
} Book;
```

Ключове поле структури – вказівник `next` на такий самий тип `Book`. Саме він утворює ланцюжок: кожен елемент «знає» адресу наступного, а останній елемент містить `NULL`, що є ознакою кінця списку. Оголошення `struct Book *next` всередині самої структури допустиме, оскільки вказівник має фіксований розмір незалежно від того, чи описана структура повністю.

Програму розділено на функції:

- `createBook()` – виділяє пам'ять під новий елемент функцією `malloc()` і заповнює його поля;
- `addBook()` – додає готовий елемент у кінець списку;
- `fillList()` – заповнює список даними про сім книг серії;
- `copyString()` – власне копіювання рядка у поле структури (замість `strcpy`);
- `countBooks()` – підраховує кількість елементів обходом списку;
- `printBook()` і `printList()` – виведення одного елемента та обхід усього списку;
- `printSummary()` – підсумкова інформація: загальна ціна, кількість сторінок, вага, середня ціна;
- `freeList()` – звільняє пам'ять під усіма елементами списку;
- `pauseBeforeExit()` – затримка перед завершенням, щоб вікно консолі не закрилося одразу.

Робота з динамічною пам'яттю:

Кожен елемент списку створюється окремим викликом `malloc(sizeof(Book))`, тому кількість книг не обмежена наперед заданим розміром масиву. Результат `malloc()` обов'язково перевіряється на `NULL`: якщо пам'ять виділити не вдалося, програма повідомляє про помилку і завершується. Наприкінці роботи функція `freeList()` проходить список і звільняє кожен елемент. Важлива деталь реалізації – адреса наступного елемента зберігається у змінну `next` до виклику `free()`, інакше після звільнення пам'яті звертатися до поля `current->next` вже не можна. Після звільнення покажчика `head` присвоюється `NULL`, а повторний виклик `countBooks()` повертає 0, що підтверджує коректність очищення.

Передавання списку у функції:

Функції, які лише читають список (`printList`, `countBooks`, `printSummary`), приймають `const Book *head` – копію адреси першого елемента. Функція `addBook()` приймає `Book **head`, тобто адресу самого покажчика: це потрібно,

щоб при додаванні першої книги вона могла змінити значення head у функції main(). Якби параметр був оголошений як Book *head, зміна вплинула б лише на локальну копію, і список залишився б порожнім.

Посилання на додаток у Git репозиторії:

Файл програми: https://github.com/blaz3xx/C-Embedded_workshops/blob/main/Books.c

Репозиторій: https://github.com/blaz3xx/C-Embedded_workshops

Лістинг програми (Books.c):

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_TITLE 64
#define MAX_LANG 16

typedef struct Book {
    char    title[MAX_TITLE];
    double  price;
    int     pages;
    char    language[MAX_LANG];
    double  weight;
    int     year;
    struct Book *next;
} Book;

void  printHeader();
void  copyString(char *dest, const char *src, int size);
Book *createBook(const char *title, double price, int pages,
                 const char *language, double weight, int year);
void  addBook(Book **head, Book *node);
void  fillList(Book **head);
int   countBooks(const Book *head);
void  printBook(const Book *book, int number);
void  printList(const Book *head);
void  printSummary(const Book *head);
void  freeList(Book **head);
void  pauseBeforeExit();

int main(void) {
    Book *head = NULL;

    printHeader();

    fillList(&head);

    if (head == NULL) {
        printf("The list is empty.\n");
        pauseBeforeExit();
        return EXIT_FAILURE;
    }

    printList(head);
    printSummary(head);
}
```

```

        freeList(&head);
        printf("\nMemory has been released. Books in the list: %d\n",
countBooks(head));

        pauseBeforeExit();

        return EXIT_SUCCESS;
}

void printHeader() {

printf("=====\n");
    printf("  Author:  Artem Cherednichenko\n");
    printf("  Task:    Harry Potter book series stored in a linked list\n");

printf("=====\n\n");
}

void copyString(char *dest, const char *src, int size) {
    int i = 0;

    while (src[i] != '\0' && i < size - 1) {
        dest[i] = src[i];
        i++;
    }

    dest[i] = '\0';
}

Book *createBook(const char *title, double price, int pages,
                const char *language, double weight, int year) {
    Book *node = (Book *)malloc(sizeof(Book));

    if (node == NULL) {
        printf("Memory allocation error.\n");
        exit(EXIT_FAILURE);
    }

    copyString(node->title, title, MAX_TITLE);
    copyString(node->language, language, MAX_LANG);
    node->price = price;
    node->pages = pages;
    node->weight = weight;
    node->year = year;
    node->next = NULL;

    return node;
}

/* додавання елемента в кінець списку */
void addBook(Book **head, Book *node) {
    Book *current;

    if (*head == NULL) {
        *head = node;
        return;
    }

    current = *head;
    while (current->next != NULL) {
        current = current->next;
    }

```

```

        current->next = node;
    }

/* заповнення списку даними */
void fillList(Book **head) {
    addBook(head, createBook("Harry Potter and the Philosopher's Stone",
                             249.00, 223, "English", 280.0, 1997));
    addBook(head, createBook("Harry Potter and the Chamber of Secrets",
                             269.00, 251, "English", 310.0, 1998));
    addBook(head, createBook("Harry Potter and the Prisoner of Azkaban",
                             299.00, 317, "English", 380.0, 1999));
    addBook(head, createBook("Harry Potter and the Goblet of Fire",
                             399.00, 636, "English", 720.0, 2000));
    addBook(head, createBook("Harry Potter and the Order of the Phoenix",
                             449.00, 766, "English", 860.0, 2003));
    addBook(head, createBook("Harry Potter and the Half-Blood Prince",
                             429.00, 607, "English", 690.0, 2005));
    addBook(head, createBook("Harry Potter and the Deathly Hallows",
                             429.00, 607, "English", 700.0, 2007));
}

/* кількість елементів у списку */
int countBooks(const Book *head) {
    int count = 0;
    const Book *current = head;

    while (current != NULL) {
        count++;
        current = current->next;
    }

    return count;
}

/* виведення одного елемента */
void printBook(const Book *book, int number) {
    printf("Book #%d\n", number);
    printf("  Title:      %s\n", book->title);
    printf("  Price:       %.2f UAH\n", book->price);
    printf("  Pages:        %d\n", book->pages);
    printf("  Language:     %s\n", book->language);
    printf("  Weight:       %.1f g\n", book->weight);
    printf("  Year:         %d\n\n", book->year);
}

/* обхід усього списку */
void printList(const Book *head) {
    const Book *current = head;
    int number = 1;

    while (current != NULL) {
        printBook(current, number);
        current = current->next;
        number++;
    }
}

/* підсумкова інформація */
void printSummary(const Book *head) {
    const Book *current = head;
    double totalPrice = 0.0;
    double totalWeight = 0.0;
    int    totalPages = 0;
    int    count = 0;

```

```

        while (current != NULL) {
            totalPrice += current->price;
            totalWeight += current->weight;
            totalPages += current->pages;
            count++;
            current = current->next;
        }

        printf("-----\n");
        printf("Books in the series: %d\n", count);
        printf("Total price: %.2f UAH\n", totalPrice);
        printf("Total pages: %d\n", totalPages);
        printf("Total weight: %.1f g\n", totalWeight);
        printf("Average price: %.2f UAH\n", totalPrice / count);
        printf("-----\n");
    }

    /* звільнення пам'яті */
    void freeList(Book **head) {
        Book *current = *head;
        Book *next;

        while (current != NULL) {
            next = current->next;
            free(current);
            current = next;
        }

        *head = NULL;
    }

    void pauseBeforeExit() {
        printf("\nPress Enter to exit...");
        getchar();
    }
}

```

Скріншот виконання додатку:

C:\Users\Admin\Desktop\KPI_3_course\C_Embedded\PR2\cmake-build-debug\Books.exe

```
=====
Author:  Artem Cherednichenko
Task:    Harry Potter book series stored in a linked list
=====
```

Book #1

```
Title:    Harry Potter and the Philosopher's Stone
Price:    249.00 UAH
Pages:    223
Language: English
Weight:   280.0 g
Year:     1997
```

Book #2

```
Title:    Harry Potter and the Chamber of Secrets
Price:    269.00 UAH
Pages:    251
Language: English
Weight:   310.0 g
Year:     1998
```

Book #3

```
Title:    Harry Potter and the Prisoner of Azkaban
Price:    299.00 UAH
Pages:    317
Language: English
Weight:   380.0 g
Year:     1999
```

Book #4

```
Title:    Harry Potter and the Goblet of Fire
Price:    399.00 UAH
Pages:    636
Language: English
Weight:   720.0 g
Year:     2000
```

Book #5

```
Title:    Harry Potter and the Order of the Phoenix
Price:    449.00 UAH
Pages:    766
Language: English
Weight:   860.0 g
Year:     2003
```

Book #6

```
Title:    Harry Potter and the Half-Blood Prince
Price:    429.00 UAH
Pages:    607
Language: English
Weight:   690.0 g
Year:     2005
```

Book #7

```
Title:    Harry Potter and the Deathly Hallows
Price:    429.00 UAH
Pages:    607
Language: English
Weight:   700.0 g
Year:     2007
```

```
-----
Books in the series: 7
```

```
Total price:    2523.00 UAH
Total pages:     3407
Total weight:    3940.0 g
Average price:   360.43 UAH
-----
```

Memory has been released. Books in the list: 0

Press Enter to exit...

Результат роботи програми (вивід консолі, скорочено):

```
=====
Author:  Cherednichenko
Task:    Harry Potter book series stored in a linked list
=====

Book #1
Title:   Harry Potter and the Philosopher's Stone
Price:   249.00 UAH
Pages:   223
Language: English
Weight:  280.0 g
Year:    1997

Book #2
Title:   Harry Potter and the Chamber of Secrets
Price:   269.00 UAH
Pages:   251
Language: English
Weight:  310.0 g
Year:    1998

...

Book #7
Title:   Harry Potter and the Deathly Hallows
Price:   429.00 UAH
Pages:   607
Language: English
Weight:  700.0 g
Year:    2007

-----
Books in the series: 7
Total price:         2523.00 UAH
Total pages:         3407
Total weight:        3940.0 g
Average price:       360.43 UAH
-----

Memory has been released. Books in the list: 0

Press Enter to exit...
```

Висновок: у результаті виконання практичного заняття №4 було розроблено консольний додаток мовою С, який зберігає інформацію про серію книг про Гаррі Поттера в однозв'язному списку та виводить її на екран. Набуто навички опису структур із вказівником на самих себе, роботи з динамічною пам'яттю через `malloc()` і `free()`, побудови та обходу зв'язаного списку, а також передавання списку у функції через подвійний вказівник. На відміну від масиву, зв'язаний список не потребує наперед відомої кількості елементів і дозволяє додавати нові записи під час виконання програми. Проєкт збережено

у локальному репозиторії та синхронізовано з віддаленим репозиторієм на GitHub.